Cary Jones - Senior Fullstack Role Questions

Can you describe a project where you designed or implemented AI-powered features, particularly for matching, recommendation, or personalization? What challenges did you face, and how did you address them?

> I have not yet had a chance to degign or implement an AI-driven solution, however, I am in conversation with someone who has a need for this type of product. I will describe how I would think about tackling a project of this nature. The premise is that there is an old teacher with many years of teaching material and books he has written who wants to use AI to allow people to ask questions where the answers are given as though from the teacher himself, but reaching a wide audience, much wider than he alone could reach.

- The implemetation would be to create a prompt instructing the AI to consume the given material and be prepared to answer subsequent questions based on the prompt. The AI itself would be something like the Grok API around which would be wrapped a web service. The frontend would receive end user requests (questions) via a web page and pass them to the backend which would communicate with the Grok API and forward the answers back to the frontend for the user to view. Some challenges I can forsee might be:
    - Getting the enourmous amount of material the teacher has compiled over the years into the prompt(s). That is less of a technical challenge and would be more time consuming, but could become technical if the prompt was split into various domains or topics resulting in multiple prompts. The domains would need to be organized in a logical way to maximize efficiency of retrieval and reference by the AI as well as ensuring the prompts are able to communicate well with each other. There could be some data science involved making sure the questions are routed to the correct prompt.
    - Ensuring the prompt is properly trained so that answers given remain strictly valid enough to satisfy the teacher's concern that answers will not deviate too far from the material or the intent of the instruction set. This could be accomplished by either training the prompt with more material (which would already be abundant) or using data science mechanisms to ensure answers given are not deviating too far. For example, SMOTE (Synthetic Minority Oversampling Technique) could be used, which addresses imbalanced data sets such as those in insurance actuarial calcuations where fraud cases need to be detected. Also establishing an F1-score could aid in determining if answers are straying from the educational intent. Overfitting could be addressed by early stopping or L1, L2 regularization depending on the amount of stray found in the answers and in what domains or ridges it may be occuring.
    - Testing answers and adjusting via above techniques would be time consuming, but likely necessary to ensure the teacher is being well represented.

Can you share an example of mentoring other engineers or improving technical practices within a team? How do you balance coding, reviewing,

and coaching?

- Several years ago I was named CTO of a small startup developing a patient medical data application meant to simplify the doctor/patient interaction during the care providing process. I was working with some very junior contractor developers who had difficulty with git branching. They had some trepidation about using rebasing and had been told by their parent company not to do it. It's true that it's a bit more complex than simply checking in code by fast-forward merging, but their inability to use it was causing issues. Each time they were done with the feature branch they were working on, they would simply merge their code into the development branch when they were done with the ticket. The problem was that potential merge conflicts were not adressed, nor were they even seen until it was time to merge the development branch into the main/master. It took hours for me to go back and resolve all the merge conflicts they had introduced. As a result, I created a branching guide, detailing for them the process of, first branching from development, then rebasing the development branch onto their feature branch often, and definitely before creating a pull request, to ensure potential merge conflicts were dealt with by them, at their level rather than being introduced into the higher branch. I taught them how to use interactive rebasing and that they have total control of the order, naming, number, and content of code in their feature branch commits before create a pull request. It wasn't easy and they were slow to pick it up, but seeing their progress and willingness to contribute to the project in a more powerful way was worth the effort.

- When things get really stuck, or sometimes just to enhance team collaboration, pair-programming can be a powerful tool. As a senior developer, this can be a chance to see how the minds of team members work. Rarely is there a completely wrong way to do something and everyone approaches things differently. Getting to know fellow team members while they're working can really help reduce frustration and even create empathy for someone when they're experiencing difficulty. Nearly every junior developer I've met has a true passion for learning technology. Sometimes you can see that the way they're thinking about the solution to a problem just hasn't matured yet. And sometimes it's apparent that they're taking what they've learning and applying it correctly, but something else is at play. There have even been a few times when I was pair-programming with a junior team mate and realized the way they were doing something was better or more efficient that the way I was used to doing it. I'm always happy to pick up new tricks if they work better.

Describe a situation where you worked closely with product managers, data scientists, or security teams to solve a complex business problem. How did you ensure the technical solution aligned with business goals?

When I worked with Fox Sports in Los Angeles in 2019, I worked closely with our chief data scientist who was in charge of building a sports stats dashboard. In addition to personally building out several modules that processed streaming live data for MLB stats, I also advised the team (based on prior successes as a Scrum master), on a way to streamline team development efforts by increased collaboration using the Scrum process as intended. I've found during my career that many companies use parts of the Scrum process that they think are useful and leave out other parts. In the end it always creates more pain to leave out those parts. As we began to implement Scrum correctly, it was slower at first and garnered lots of complaints having to relearn certain things, but the pace began to pick up and team members started to express feeling much lower stress levels and an easier ability to think about, break down into manageable pieces, and process

tasks on the board. Before this, the project has just seemed to be stuck no matter how hard we worked, and management was starting to get very frustrated. After increased collaboration the project seemed to almost magically break free and was able to move forward. Technically, I think we were doing all the right things, but our interactions with each other needed an overhaul.

How would you architect a system that needs to handle millions of users with AI-driven matchmaking, ensuring reliability, performance, and maintainability?

This kind of system undoubtedly needs sound logic, containerization, high availabilty and reliable failover. I think 4 main things are in order:

- Matching: in terms of application logic, a graph is an extremely useful tree datastructure in determining who may be aware of or who may want to get to know who inside a mesh topology. Opposingly, it can also provide useful logic in confidently preventing likely unwanted matches. Neo4j is an excellent technology that has taken data storage to a new level in terms of relationship and dependency mapping. These kinds of relational representations can be stored in relational databases but not without a lot of extra pain and certainly pages of SQL logic to construct even some of the more simple relationships. Complex ones can become unreadable and unmanageable.
- Modularization: areas of concern need to be put into logical domains and kept small. They also need the ability to communicate with each other in a meaningful way. Some modules will be able to keep their data private, others need to share more freely. I think event-driven design can be very powerful. Often using a messaging system such as RabbitMQ to offload data processing from one module to another once the module has done it's part keeps the modules from becoming fused together in the dependency flow. This also allows for rapid, timely flow of data without fear of loss or failure to process events. Only messages that are ack'd will be taken off the queue ensuring that in the event of a failure or error, the data (or reference to it) remains in the queue for processing until it is complete once services have restarted. Any application that is intended to provide messaging services, such as a chat or member-to-member messaging is an obvious candidate for a backend messaging queue as this was its original intended use. Event-driven services are exciting because they operate very fast and can handle seeminly unlimited amounts of messages/events. Services like RabbitMQ and Redis can themselves be setup as HA with sentinels to address failover as well as providing deep resources for processing floods of incoming requests.
- Containerization: modules or domains often nicely fit into containers, with many predefined services such as Redis, Nginx, Neo4j, and many others (including custom services) being prepackaged as docker containers just waiting for env vars or config changes to be applied. As metioned below, containers can be orchestrated to provide for streamlined pipelining and deployment.
- Deployment: making these services highly available to the public has always been a challenge. In the past, running on physical hardware (bare metal), the required hot-swapping of drives, memory modules, scsi components, and even entire servers was common under heavy load increases. Today, services like Kubernetes excel at packaging encapsulated functionality into pods that are pre-networked together and can communicate internally without extra networking overhead. Layered containerization surpasses the old virtual machine model leveraging hypervisor management of resource striping to direct the use of data center (or on-prem) computing resources in a much more efficient way. Ephemeral pods can die and restart via container orchestration, and as long as there is no fundamental issue with the application code, experience little to no downtime. To service millions of users, multiple, dozens, or even hundreds or thousands

of pods, nodes, and clusters can be deployed and load-balanced to ensure applications are capable of handling the sheer volume without breaking under the strain.

Tell me about a time you made a technical decision that significantly impacted the direction of a product or platform? How did you evaluate options and get buy-in from stakeholders?

When I worked at Disney recently, I worked with the human resources process automation team. The idea was to track time worked via the Jira API and match it to the creation/deletion of human resource requistioning and also process employee/contractor payments. The various modules were linked together in a hub-and-spoke pattern and thus, were loosely coupled. As the team worked together, it became more and more apparent that what we needed was a web service with an internal customer facing frontend. The backend would sit atop the hub and manage the various spoke services that data was being pulled from and store them in an API that could be consumed by the frontend but also by other business units for whom the frontend may make less sense. Isay "store in an API", but in reality we were using SQL queries to PostgreSQL with a level of complexity that was mind-bending. I believed we would benefit tremendously from unwinding all that and modularizing it. We all knew what needed to be done and the longer we held out the more painful and obvious it became. In my spare time, I began developing some backend modeling of the various components we were working with. The models were Python objects and were being prepped as the interfaces for the spoke services. One day I was finally able to demonstrate the models to the team and got buy-in from everyone that this effort might be easier than we thought. I think my passion for web service development was contagious and my extra efforts allowed the team to see that this really could be possible after all.

Do you have other interviews going on?

Yes, I just completed a 2nd meet-and-greet interview with the technical manager at another company.

What is your interview availability over the next 2 weeeks for multiple interviews?

I am available for interviews in the late evenings, mostly M-F, but can make exceptions.

Why are you a strong fit for this role?

I believe that besides bringing 20 years of experience in the software engineering field, I am the type of developer who is a people person. I love interacting with others and thrive best where I can work in and help foster a collaborative environment. I have had many junior and senior roles, including running my own company and servicing clients directly. I know what it's like to be the new guy, observing and learning with passion, and I know what it's like to feel the pressure of managing resources so that a team can succeed, the most notable of which are finances. I feel that I've seen a broad spectrum of what the technology industry is, but I know there's always more to see and experience and that always excites and drives me on.

Why are you looking for a new role?

During the past couple of years, interests rates have been high and I think many companies have been trying to navigate that the best they can. That being said, it's been more difficult to find employment over that past 2 years than I can remember at any other point in my career. For some time during that period, I chose to focus on investing and learning about options trading. I wasn't successful in the sense of making a lot of money, but the deep-dive into calculus and statistics have created some powerful tools to add to the repertoire of skills I bring to the table. I feel the time has come to dive back into the software development industry and continue to contribute where I feel my best abilities lie. I am excited to again be engaged with others in the creation of products and systems that change the way we live our lives.